A

APEX

Manual técnico

Arquitectura, estructura del código, base de datos, reglas de negocio y cálculo, seguridad técnica, pruebas y guía para extender el sistema.

Dirigido a: desarrolladores y personal de soporte técnico

APEX · Sistema de control de cobros
Versión 1.3 · octubre de 2026

Desarrollado por
Ing. Antony Pérez

Contenido

1. Visión general
   - 1.1 Tecnologías
   - 1.2 Principios de diseño
2. Arquitectura
   - 2.1 Ciclo de una petición
   - 2.2 Enrutamiento
   - 2.3 Estructura del código
   - 2.4 Rutas
3. Reglas de negocio y cálculos
   - 3.1 Plan de cuotas
   - 3.2 Aplicación de pagos: Credito::recalcular()
   - 3.3 Renovación: Credito::renovar()
   - 3.4 Mora
   - 3.5 Calificación del cliente: Cliente::calificaciones()
   - 3.6 Promesas de pago
   - 3.7 Caja
   - 3.8 Rentabilidad: Reporte::rentabilidad()
   - 3.9 Cartera por cobrador
   - 3.10 Orden de la ruta: Cliente::ordenarRuta()
   - 3.11 Trabajo sin conexión
4. Seguridad técnica
5. Frontend
6. Generación de Excel
7. Pruebas
8. Cómo extender el sistema
   - 8.1 Agregar un módulo

8.2 Convenciones

9. Base de datos

9.1 Relaciones principales

9.2 Diccionario de datos

1. Visión general

1.1 Tecnologías

Backend	PHP 8.1+ con `declare(strict_types=1)`. Arquitectura MVC propia, sin framework ni Composer.
Base de datos	MySQL / MariaDB con InnoDB, utf8mb4 y llaves foráneas. Acceso con PDO y consultas preparadas.
Frontend	HTML renderizado en el servidor, Bootstrap 5.3, Bootstrap Icons y Chart.js 4 (CDN jsDelivr), y JavaScript propio sin dependencias (`assets/js/app.js`, `assets/js/offline.js`).
Sin conexión	Service worker (`sw.js`) con Cache Storage para consultar páginas guardadas; IndexedDB para la cola de cobros hechos sin señal; manifiesto web (`manifest.webmanifest`) para instalarlo como aplicación.
Excel	Generador .xlsx propio (`app/services/Xlsx.php`) que arma el contenedor ZIP con `gzdeflate`; no requiere la extensión zip.
Servidor	Apache 2.4 con `mod_rewrite` y `mod_headers`.

1.2 Principios de diseño

- **Un solo punto de entrada** (`index.php`); todo lo demás está bloqueado por `.htaccess`.
- **El servidor es la fuente de verdad:** toda validación y permiso se aplica en PHP; el JavaScript solo mejora la experiencia.
- **Integridad contable:** los pagos nunca se borran (se anulan) y los saldos se recalculan a partir de los pagos vigentes, de forma que el plan siempre cuadra con la caja.
- **Trazabilidad:** toda acción relevante queda en la auditoría.
- **Sin conexión, sin atajos:** lo registrado sin señal pasa por las mismas validaciones del servidor al sincronizar; el celular nunca es fuente de verdad.

2. Arquitectura

2.1 Ciclo de una petición



2.2 Enrutamiento

La URL `/controlador/accion/param1/param2` se traduce a `{Controlador}Controller::{accion}(param1, param2)`. Los guiones se convierten a camelCase (`caja/anular-gasto/7` → `CajaController::anularGasto('7')`). La ruta vacía va a `dashboard/index`.

El router (`app/core/App.php`) solo ejecuta métodos **públicos, no estáticos, declarados en el propio controlador** y que no empiecen por `__`; valida el nombre con `^[a-z0-9-]+$` y exige el número mínimo de parámetros. Cualquier otro caso responde 404. Las excepciones de base de datos y las inesperadas se registran en el log y muestran una página de error genérica (o el detalle si `APP_DEBUG=true`).

2.3 Estructura del código

```
app/
├── bootstrap.php          Autoload PSR-4 (App\ → app/), .env, zona horaria, errores, cabeceras y sesión
├── config/config.php      Configuración leída de .env
├── core/
│   ├── App.php            Router
│   ├── Controller.php     Base: view(), redirect(), back(), json(), requireLogin(), requireRole(),
│   │                      requirePost() (CSRF), input(), withOld()
│   ├── Model.php          Base: find(), all(), create(), update(), delete(), fetchAll/One/Value()
│   ├── Database.php       Conexión PDO única (errores como excepción, sin emulación de preparadas)
│   ├── Auth.php           Sesión del usuario y su revalidación por petición
│   ├── Security.php       Cabeceras HTTP, nonce CSP, sesión endurecida, IP, URL interna
│   ├── Password.php       Política de contraseñas y hash Argon2id
│   ├── Config.php, Env.php
├── controllers/           Un controlador por módulo
├── models/                Acceso a datos y reglas de negocio
├── services/              Xlsx, ReporteExcel, Respaldo
├── helpers/functions.php  e(), url(), money(), parse_money(), fecha(), csrf_field(), flash(), badge()…
└── views/                 Vistas por módulo, layouts (main, auth, print, documento) y parciales
```

2.4 Rutas

Controlador	Acciones (GET salvo que se indique POST)	Acceso
Auth	login (GET/POST), logout (POST)	Público
Dashboard	index	Ambos
Ruta	index, hoja (hoja de ruta imprimible), ordenar (POST, JSON)	Ambos (ruta propia)
Sincronizar	estado (JSON: sesión y token CSRF), operacion (POST, JSON)	Ambos
Clientes	index, crear, guardar (POST), editar/{id}, actualizar/{id} (POST), ver/{id}, eliminar/{id} (POST, admin), asignar (POST, admin)	Ambos (cartera propia)
Creditos	index, ver/{id}, estado/{id}, pazysalvo/{id}, excel/{id}; crear, renovar/{id}, simular, guardar (POST), anular/{id} (POST): admin	Ambos / admin
Pagos	index, guardar (POST), recibo/{id}; anular/{id} (POST): admin	Ambos / admin
Gestiones	guardar (POST)	Ambos (cartera propia)
Caja	index, gasto (POST), anular-gasto/{id} (POST); cerrar (POST): admin	Ambos / admin
Cobradores	index (admin), ver/{id} (el cobrador solo el suyo), trasladar/{id} (POST, admin)	Ambos / admin
Reportes	index, exportar/{tipo}	Admin
Usuarios	index, crear, guardar, editar/{id}, actualizar/{id}, estado/{id}, eliminar/{id}	Admin

Controlador	Acciones (GET salvo que se indique POST)	Acceso
Configuracion · Auditoria · Respaldo	index, guardar (POST) · index · index, descargar (POST)	Admin
Perfil	index, clave (POST)	Ambos

3. Reglas de negocio y cálculos

3.1 Plan de cuotas

`Credito::calcularPlan()` (estático, también lo usa el simulador):

total = round(monto × (1 + interés / 100))
cuota = ceil(total / n / 100) × 100; si total − cuota × (n − 1) ≤ 0 → cuota = floor(total / n)
última cuota = total − cuota × (n − 1)

- **Fechas:** diaria +1 día (si `excluir_domingos` y cae domingo, +1); semanal +7; quincenal +15; mensual mismo día del mes siguiente con tope en el último día (`sumarMes()` conserva el día original: 31 ene → 28 feb → 31 mar).
- La creación es transaccional: crédito + código `CR-%05d` + cuotas.

3.2 Aplicación de pagos: Credito::recalcular()

Tras registrar o anular un pago, el sistema **reconstruye** el estado del crédito desde cero: pone en 0 lo abonado de todas las cuotas y redistribuye todos los pagos vigentes en orden cronológico, de la cuota más antigua a la más nueva. La fecha de pago de cada cuota es la del pago que la completó. Luego actualiza `total_pagado`, `saldo = total − Σpagos` y el estado (`pagado` si saldo ≤ 0). Esto garantiza que anular un pago antiguo produce exactamente el mismo resultado que si nunca se hubiera registrado.

`Pago::registrar()` bloquea el crédito con `SELECT … FOR UPDATE` dentro de la transacción y rechaza montos mayores al saldo, evitando que dos pagos simultáneos excedan la deuda.

3.3 Renovación: Credito::renovar()

En una sola transacción: bloquea el crédito anterior, valida `monto > saldo`, inserta el crédito nuevo con `renovado_de`, `saldo_renovado` y `neto_entregado = monto − saldo`, e inserta en el anterior un pago con `metodo = 'renovacion'` por el saldo, que lo deja pagado. Los pagos de renovación se excluyen de todo cálculo de recaudo, caja y rentabilidad, y no se pueden anular directamente: anular el crédito nuevo anula ese movimiento y el anterior recupera su saldo.

3.4 Mora

Una cuota está vencida si `estado <> 'pagada' AND fecha_vencimiento < CURDATE()` en un crédito activo. Días de mora = días desde la cuota vencida más antigua.

3.5 Calificación del cliente: Cliente::calificaciones()

Se evalúan todas las cuotas de créditos no anulados que estén pagadas o vencidas. A tiempo: pagada con `fecha_pago ≤ fecha_vencimiento + 1 día`.

puntualidad = a_tiempo / (a_tiempo + tarde + vencidas) · mora = días de la cuota vencida más antigua (crédito activo)

N si hay menos de 3 cuotas evaluadas · A: ≥ 90 % y mora ≤ 2 · B: ≥ 75 % y mora ≤ 7 · C: ≥ 50 % y mora ≤ 30 · D: el resto

3.6 Promesas de pago

Una gestión con `resultado = 'promesa'` está **cumplida** si la suma de pagos vigentes (sin renovaciones) entre `created_at` de la gestión y el final del día `fecha_promesa` es mayor o igual a `monto_promesa` (o a 1 si no se indicó monto); **incumplida** si la fecha ya pasó; si no, **pendiente**. Se calcula en la consulta, sin almacenarse.

3.7 Caja

a_entregar(día) = Σ pagos en efectivo del cobrador en el día − Σ gastos vigentes del día
en_mano = Σ efectivo histórico − Σ gastos históricos − Σ entregado en cierres

Un cierre guarda una foto de los valores (`UNIQUE(cobrador_id, fecha)`; cerrar de nuevo lo reemplaza). Si los movimientos del día cambian después, la pantalla lo marca como desactualizado. Si `a_entregar < 0` y se entrega 0, se registra como reembolso (`entregado = a_entregar`, diferencia 0).

3.8 Rentabilidad: Reporte::rentabilidad()

interés cobrado = Σ pago × (total − monto) / total · capital recuperado = recaudo − interés cobrado
utilidad = interés cobrado − gastos de ruta · margen = utilidad / recaudo

3.9 Cartera por cobrador

La cartera se calcula por el cobrador **actualmente asignado** al cliente (`clientes.cobrador_id`); el recaudo, por quien recibió cada pago (`pagos.cobrador_id`). Así, al trasladar un cliente su saldo pasa al nuevo cobrador, pero lo ya cobrado sigue a nombre de quien lo cobró.

3.10 Orden de la ruta: Cliente::ordenarRuta()

Cada cliente tiene `orden_ruta` (posición en la ruta de su cobrador; NULL = al final, por `id`). `ordenarRuta($cobradorId, $ids)` bloquea con `FOR UPDATE` los clientes del cobrador en su orden actual, toma solo los ids recibidos que le pertenecen y los reubica en los **huecos que ocupaban**, de menor a mayor. Así, ordenar la vista filtrada (por ejemplo, solo los clientes con cobro hoy) no mueve a los demás. Luego renumera 1…n. Al reasignar un cliente a otro cobrador, su `orden_ruta` vuelve a NULL y entra al final. `Credito::ruta()` ordena por cobrador, `orden_ruta IS NULL`, `orden_ruta`, cliente y crédito.

3.11 Trabajo sin conexión

Consultar: `sw.js`

- **Estáticos** (`assets/` y CDN): primero caché, luego red. Las URL llevan `?v=filemtime` (`asset()`), así que una actualización genera URL nuevas.
- **Páginas HTML** (navegación GET): primero red, guardando copia en la caché `apex-paginas` (máximo 300). Sin red, o si la red tarda más de 8 s y hay copia, se sirve la copia con una franja "Sin conexión · copia guardada el …" y la meta `apex-copia=1`. Sin copia exacta se busca la misma ruta sin parámetros; si no hay, `assets/offline.html` lista las páginas guardadas.
- **Nunca se guardan:** `auth/*`, `sincronizar/*`, `respaldo/*`, descargas (`excel`, `exportar`, `descargar`), `simular`, `ordenar` y `ayuda/ver`. Las respuestas con `X-Apex-Copia: no` (avisos flash o "Ver recibo" de un solo uso) se vuelven a pedir limpias antes de guardarlas.
- `offline.js` pide al service worker precargar (mensaje `precache`) las pantallas principales cada 30 minutos y, desde la ruta, la ficha de cada cliente y crédito del día. Al cerrar sesión envía `limpiar`, que borra `apex-paginas`.

Registrar: `assets/js/offline.js`

- Solo en la Ruta de cobro. Intercepta los formularios de pago y gestión: cada operación lleva un `op_uuid` generado con `crypto.getRandomValues`. Si no hay red o el envío no responde en 12 s, la operación (tipo, datos, hora local `creado_en`, usuario) se guarda en IndexedDB (`apex-offline` → `cola`, con secuencia creciente).
- El orden de la ruta sin señal se guarda como operación `orden` (la última reemplaza a la anterior).
- Al volver la conexión (evento `online`, al cargar y periódicamente) se llama a `sincronizar/estado` para validar la sesión y obtener un token CSRF fresco, y luego se envían las operaciones del mismo usuario **una a una, en orden de secuencia**, a `sincronizar/operacion`. Se detiene ante un error de red; las rechazadas (`definitivo:true`) quedan "por revisar" para reintentar o descartar.
- Las operaciones de otro usuario guardadas en el mismo equipo no se envían ni se muestran.

Servidor: `SincronizarController`

- **Idempotencia:** `OperacionOffline::reservar()` inserta el uuid en `operaciones_offline` dentro de la misma transacción que crea el pago o la gestión (`Pago::registrar()`, `Gestion::registrar()`). Si el uuid ya existe (PK duplicada) se responde `{ok:true, duplicado:true}` con el recibo original. Los formularios en línea de la ruta también envían `op_uuid`, así que un reenvío por mala señal tampoco duplica.
- **Validación:** se reaplican todas las reglas: crédito activo y de la cartera del usuario, valor ≤ saldo, método válido, fecha de promesa.
- **Fecha del pago:** `creado_en` se acepta si no es futura (tolerancia 10 min) ni anterior a 7 días; si no, se usa la hora del servidor. Si `Caja::diaCerrado()` para ese cobrador y día, se usa la hora actual y la nota indica cuándo se cobró. La nota siempre empieza por "Registrado sin conexión".
- **Respuestas:** `{ok:true}` · `{ok:false, definitivo:true, mensaje}` (422, rechazo de negocio) · `{redirect}` (sesión vencida).

4. Seguridad técnica

Amenaza	Control implementado
Inyección SQL	PDO con consultas preparadas y `ATTR_EMULATE_PREPARES=false`. Los únicos valores interpolados son enteros convertidos con `(int)` o listas fijas del código.
XSS	Toda salida pasa por `e()` (`htmlspecialchars` ENT_QUOTES). JSON embebido con `js_json()` (flags HEX). CSP con nonce por petición: `script-src 'self' 'nonce-…' cdn.jsdelivr.net`, sin `unsafe-inline` ni manejadores `on*`.
CSRF	Token por sesión en todo POST (`requirePost()` con `hash_equals`); el token se regenera al iniciar sesión. Las acciones que modifican datos solo aceptan POST (405 en otro caso). Cookie SameSite=Lax.
Control de acceso / IDOR	`requireRole()` por controlador o acción y verificación de pertenencia a la cartera (`Auth::scopeCobrador()` , `cargar()` de cada controlador). Los campos sensibles (cobrador, estado, fecha del pago, quién recibe) se ignoran si los envía un cobrador.
Fuerza bruta	`IntentoLogin` en BD: N fallos por usuario+IP, 2N por usuario y 4N por IP en la ventana. Mismo mensaje para usuario inexistente; el hash ficticio iguala el tiempo de respuesta.
Sesión	HttpOnly, SameSite, Secure con HTTPS, `use_strict_mode` , nuevo ID al ingresar, expiración por inactividad y absoluta, huella del navegador, y revalidación en BD en cada petición (activo, rol, `sesion_version`).
Contraseñas	Argon2id (m=64 MB, t=3) con rehash transparente de hashes antiguos; política de complejidad (`Password::validar()`); cambio obligatorio (`debe_cambiar_clave`).
Exposición	`.htaccess` bloquea app/, database/, storage/, tests/, docs/, archivos ocultos y extensiones sensibles; no ejecuta otro .php distinto de index.php; sin listado de carpetas; sin X-Powered-By; métodos limitados a GET/POST/HEAD.
Redirección abierta	`back()` solo redirige si el Referer coincide en esquema, host, puerto y ruta base (`Security::isInternalUrl()`).
Datos de entrada	`input()` descarta arreglos inesperados; `parse_money()` estricto (null ante negativos, decimales o texto); longitudes recortadas; fechas validadas con `DateTime::createFromFormat` .
Excel	El texto se escribe como cadena literal (inlineStr): nunca se evalúa como fórmula.
Datos en el celular	Las copias de páginas se borran al cerrar sesión; descargas, ingreso y respaldos nunca se guardan. La cola sin conexión se filtra por usuario y solo se envía con una sesión válida del mismo usuario; el servidor revalida permisos, CSRF y reglas de negocio en cada operación.

Ver también el **Manual de seguridad**.

5. Frontend

- **Layouts:** `main` (aplicación), `auth` (ingreso), `print` (recibo 80 mm), `documento` (carta: estado de cuenta y paz y salvo).
- **Scripts por vista:** se capturan en `$scripts` con `ob_start()` y se imprimen al final del layout con `nonce="<?= csp_nonce() ?>"`.
- **app.js** ofrece: formato de miles en campos `data-money`, medidor de contraseña (`data-strength`), notificaciones (`apexToast()`), diálogo de confirmación (`apexConfirm()` y formularios con `data-confirm`, `data-confirm-title`, `data-confirm-btn`, `data-confirm-tone`, `data-confirm-input`), modales de pago y gestión, cálculo en vivo del cierre de caja y prevención de doble envío.
- **Mensajes flash:** `flash($tipo, $mensaje)` en el controlador; el parcial `views/partials/flashes.php` los muestra como notificaciones (o en línea en el login).
- **offline.js**: indicador de conexión en la barra superior, cola de cobros y gestiones sin señal, sincronización, recuadro "Guardado en este celular", precarga de pantallas y aviso "Esta acción necesita internet" en los formularios que no se pueden enviar sin señal. Arrastrar y soltar del orden de ruta (en la vista `ruta/index`).
- **Layout** `documento` también se usa para la hoja de ruta (`ruta/hoja`), con botón de imprimir / guardar PDF.
- **Prohibido:** `confirm()`, `prompt()`, `alert()` y atributos `onclick` (los bloquea la CSP).

6. Generación de Excel

`Xlsx` produce un libro OOXML válido: `[Content_Types].xml`, relaciones, propiedades del documento, `workbook.xml` (con nombres definidos para el autofiltro y los títulos de impresión), `styles.xml` y una hoja por pestaña. Los estilos se registran bajo demanda y se deduplican (`style()` devuelve el índice de `cellXfs`). Las fechas se escriben como número de serie de Excel con formato.

`ReporteExcel::hoja($nombre, $columnas, $filas, $opciones)` aplica el diseño corporativo. Cada columna define `label`, `key`, `type` (`text`, `money`, `int`, `percent`, `date`, `datetime`, `badge`), `width` y `total` (`sum`, `count`, `avg` o un texto). Opciones: `kpis`, `rowTone`, `badgeTone`, `titulo`, `nota`, `vacio`. Los totales usan `SUBTOTAL`, que respeta los filtros.

```
(new ReporteExcel('Título', 'Periodo …'))
    ->hoja('Hoja', [
        ['label' => 'Cliente', 'key' => 'cliente', 'width' => 30, 'total' => 'TOTAL'],
        ['label' => 'Valor', 'key' => 'monto', 'type' => 'money', 'total' => 'sum'],
    ], $filas, ['kpis' => [['Total', $total, 'money', 'success']]])
    ->descargar('nombre_archivo');
```

7. Pruebas

Archivo	Alcance
`tests/seguridad.php`	93 pruebas de caja negra por HTTP y verificación en BD: exposición, cabeceras, autenticación, sesiones, contraseñas, control de acceso e IDOR, CSRF, inyección, XSS, validación, Excel, router, caja, gestiones, renovaciones, documentos, respaldo y manuales. Genera `tests/resultado_seguridad.md`.
`tests/prueba_funcional.php`	32 pasos del flujo de negocio completo por HTTP, incluida la validación estructural de cada .xlsx.
`tests/verificar_produccion.php`	Verificación de solo lectura del entorno y de la integridad de los datos.

Las pruebas crean datos identificados (usuarios `sectest*`, cédulas `999000…`) y los eliminan al terminar. Se validaron con pruebas de mutación: al desactivar protecciones, las pruebas correspondientes fallan.

8. Cómo extender el sistema

8.1 Agregar un módulo

1. Crea la tabla con una migración nueva `database/migracion_vX_nombre.sql` (usa `IF NOT EXISTS`) y agrégala también al final de `schema.sql`.
2. Crea el modelo en `app/models/` extendiendo `App\Core\Model`, con `$table` y `$fillable`.
3. Crea el controlador en `app/controllers/NombreController.php`; llama a `requireLogin()` o `requireRole('admin')`, usa `requirePost()` en toda acción que modifique datos y verifica la pertenencia a la cartera si aplica.
4. Crea las vistas en `app/views/nombre/`; escapa todo con `e()` y usa `csrf_field()` en los formularios.
5. Registra los eventos con `Auditoria::log()` y agrega la acción a `Auditoria::acciones()`.
6. Agrega la opción al menú en `views/layouts/main.php` (con los roles permitidos).
7. Agrega pruebas en `tests/seguridad.php` (permisos, IDOR, validación) y ejecuta las tres suites.

8.2 Convenciones

- Dinero: `DECIMAL(14,2)` en BD, sin decimales en la interfaz; entrada con `parse_money()` y salida con `money()`.
- Fechas: `Y-m-d` en BD; mostrar con `fecha()` y `fecha_larga()`.
- Textos de la interfaz en español de Colombia; código y comentarios en español.
- Nunca borrar movimientos de dinero: anular con motivo.

9. Base de datos

9.1 Relaciones principales

Tabla	Columna	Referencia a	Al eliminar el registro referenciado
`clientes`	`cobrador_id`	`usuarios`	Queda sin cobrador (NULL)
`clientes`	`created_by`	`usuarios`	NULL
`asignaciones`	`cliente_id`	`clientes`	Se borra el historial (CASCADE)
`asignaciones`	`cobrador_anterior_id / cobrador_nuevo_id / usuario_id`	`usuarios`	NULL
`creditos`	`cliente_id`	`clientes`	Se impide borrar el cliente (RESTRICT)
`creditos`	`renovado_de`	`creditos`	Referencia lógica (sin llave foránea)
`creditos`	`created_by`	`usuarios`	NULL
`cuotas`	`credito_id`	`creditos`	Se borran las cuotas (CASCADE)
`pagos`	`credito_id`	`creditos`	Se borran los pagos (CASCADE)
`pagos`	`cobrador_id / anulado_por`	`usuarios`	NULL
`gestiones`	`credito_id`	`creditos`	CASCADE
`gestiones`	`usuario_id`	`usuarios`	NULL
`gastos`	`cobrador_id`	`usuarios`	CASCADE
`gastos`	`registrado_por`	`usuarios`	NULL
`cierres`	`cobrador_id`	`usuarios`	CASCADE
`cierres`	`recibido_por`	`usuarios`	NULL
`auditoria`	`usuario_id`	`usuarios`	NULL
`operaciones_offline`	`usuario_id`	`usuarios`	NULL

En la operación normal nunca se borran créditos ni pagos: se anulan. Los borrados en cascada solo ocurren con las herramientas de mantenimiento (datos de prueba y limpieza para producción).

9.2 Diccionario de datos

Generado a partir de la estructura real de la base de datos. En los campos ENUM, los valores permitidos aparecen en la descripción.

`usuarios`

Administradores y cobradores con acceso al sistema.

Columna	Tipo	Nulo	Clave / relación	Descripción
`id`	INT UNSIGNED	No	PK · autoincremental	Identificador interno
`nombre`	VARCHAR(120)	No		Nombre completo
`cedula`	VARCHAR(20)	Sí	Única	Cédula (obligatoria para cobradores)
`telefono`	VARCHAR(20)	Sí		Teléfono de contacto
`email`	VARCHAR(120)	Sí		Correo; también sirve para iniciar sesión
`usuario`	VARCHAR(50)	No	Única	Nombre de usuario para iniciar sesión (minúsculas)
`password`	VARCHAR(255)	No		Hash Argon2id de la contraseña (nunca la contraseña)
`rol`	ENUM	No		admin o cobrador
`zona`	VARCHAR(100)	Sí		Ruta o sector que cubre el cobrador
`meta_diaria`	DECIMAL(14, 2)	No		Meta de recaudo diario (0 = lo que vence en el día)
`activo`	BOOLEAN	No		1 puede ingresar · 0 desactivado
`debe_cambiar_clave`	BOOLEAN	No		1 obliga a cambiar la contraseña en el próximo ingreso
`clave_actualizada_at`	DATETIME	Sí		Último cambio de contraseña hecho por el usuario
`sesion_version`	INT UNSIGNED	No		Se incrementa para invalidar todas las sesiones abiertas
`ultimo_acceso`	DATETIME	Sí		Último inicio de sesión
`created_at`	TIMESTAMP	No		Fecha y hora de registro
`updated_at`	TIMESTAMP	Sí		Fecha y hora de la última modificación

`clientes`

Deudores: datos personales, contacto, ubicación, referencia y cobrador asignado.

Columna	Tipo	Nulo	Clave / relación	Descripción
`id`	INT UNSIGNED	No	PK · autoincremental	Identificador interno
`cedula`	VARCHAR(20)	No	Única	Cédula (única)
`nombres`	VARCHAR(80)	No		Nombres
`apellidos`	VARCHAR(80)	No		Apellidos
`email`	VARCHAR(120)	Sí		Correo (opcional)
`telefono1`	VARCHAR(20)	No		Teléfono principal

Columna	Tipo	Nulo	Clave / relación	Descripción
`telefono2`	VARCHAR(20)	Sí		Teléfono secundario
`direccion`	VARCHAR(180)	No		Dirección de residencia
`barrio`	VARCHAR(100)	Sí		Barrio
`ciudad`	VARCHAR(80)	Sí		Ciudad
`fecha_nacimiento`	DATE	Sí		Fecha de nacimiento
`ocupacion`	VARCHAR(100)	Sí		Ocupación o actividad
`negocio_nombre`	VARCHAR(120)	Sí		Nombre del negocio o lugar de trabajo
`negocio_direccion`	VARCHAR(180)	Sí		Dirección del negocio o trabajo
`referencia_nombre`	VARCHAR(120)	Sí		Nombre de la referencia personal
`referencia_telefono`	VARCHAR(20)	Sí		Teléfono de la referencia
`referencia_parentesco`	VARCHAR(50)	Sí		Parentesco o relación de la referencia
`observaciones`	TEXT	Sí		Notas para el cobro
`cobrador_id`	INT UNSIGNED	Sí	→ usuarios.id	Cobrador responsable actual (define a qué cartera pertenece)
`orden_ruta`	INT UNSIGNED	Sí	Índice (cobrador_id, orden_ruta)	Posición en la ruta del cobrador (NULL = al final)
`estado`	ENUM	No		activo · inactivo · bloqueado (no prestar)
`created_by`	INT UNSIGNED	Sí	→ usuarios.id	Usuario que lo registró
`created_at`	TIMESTAMP	No		Fecha y hora de registro
`updated_at`	TIMESTAMP	Sí		Fecha y hora de la última modificación

`asignaciones`

Historial de asignaciones y traslados de clientes entre cobradores.

Columna	Tipo	Nulo	Clave / relación	Descripción
`id`	INT UNSIGNED	No	PK · autoincremental	Identificador interno
`cliente_id`	INT UNSIGNED	No	→ clientes.id	Cliente reasignado
`cobrador_anterior_id`	INT UNSIGNED	Sí	→ usuarios.id	Cobrador anterior (NULL = sin cobrador)
`cobrador_nuevo_id`	INT UNSIGNED	Sí	→ usuarios.id	Cobrador nuevo (NULL = sin cobrador)
`usuario_id`	INT UNSIGNED	Sí	→ usuarios.id	Usuario que hizo el cambio
`motivo`	VARCHAR(255)	Sí		Motivo del cambio
`created_at`	TIMESTAMP	No		Fecha y hora de registro

`creditos`

Préstamos: condiciones, totales, saldo, estado y renovaciones.

Columna	Tipo	Nulo	Clave / relación	Descripción
`id`	INT UNSIGNED	No	PK · autoincremental	Identificador interno
`codigo`	VARCHAR(20)	No	Única	Código visible CR-00001
`cliente_id`	INT UNSIGNED	No	→ clientes.id	Cliente deudor
`renovado_de`	INT UNSIGNED	Sí		Crédito anterior refinanciado con este
`monto`	DECIMAL(14, 2)	No		Capital prestado
`saldo_renovado`	DECIMAL(14, 2)	No		Saldo del crédito anterior descontado en la renovación
`neto_entregado`	DECIMAL(14, 2)	Sí		Efectivo entregado al cliente (monto – saldo renovado)
`interes`	DECIMAL(6, 2)	No		Porcentaje de interés total del plazo
`total`	DECIMAL(14, 2)	No		Capital + interés
`num_cuotas`	SMALLINT UNSIGNED	No		Número de cuotas
`valor_cuota`	DECIMAL(14, 2)	No		Valor de cada cuota (la última ajusta)
`frecuencia`	ENUM	No		diaria · semanal · quincenal · mensual
`excluir_domingos`	BOOLEAN	No		1 = las cuotas diarias no caen en domingo
`fecha_inicio`	DATE	No		Fecha del desembolso
`fecha_primera_cuota`	DATE	No		Vencimiento de la primera cuota
`fecha_fin`	DATE	No		Vencimiento de la última cuota
`total_pagado`	DECIMAL(14, 2)	No		Suma de pagos vigentes (recalculada)
`saldo`	DECIMAL(14, 2)	No		total – total_pagado (recalculado)
`estado`	ENUM	No		activo · pagado · anulado
`observaciones`	TEXT	Sí		Notas del crédito
`motivo_anulacion`	VARCHAR(255)	Sí		Motivo si fue anulado
`created_by`	INT UNSIGNED	Sí	→ usuarios.id	Usuario que lo creó
`created_at`	TIMESTAMP	No		Fecha y hora de registro
`updated_at`	TIMESTAMP	Sí		Fecha y hora de la última modificación

`cuotas`

Plan de pagos de cada crédito, con lo abonado y su estado.

Columna	Tipo	Nulo	Clave / relación	Descripción
`id`	INT UNSIGNED	No	PK · autoincremental	Identificador interno
`credito_id`	INT UNSIGNED	No	→ creditos.id	Crédito al que pertenece
`numero`	SMALLINT UNSIGNED	No		Número de la cuota (1..n)
`fecha_vencimiento`	DATE	No		Fecha en que vence
`valor`	DECIMAL(14, 2)	No		Valor de la cuota
`abonado`	DECIMAL(14, 2)	No		Valor cubierto por los pagos (recalculado)
`estado`	ENUM	No		pendiente · parcial · pagada
`fecha_pago`	DATE	Sí		Fecha del pago que la completó

`pagos`

Pagos recibidos (y movimientos de renovación). Nunca se borran: se anulan.

Columna	Tipo	Nulo	Clave / relación	Descripción
`id`	INT UNSIGNED	No	PK · autoincremental	Identificador interno
`recibo`	VARCHAR(20)	No	Única	Número de recibo RC-000001
`credito_id`	INT UNSIGNED	No	→ creditos.id	Crédito abonado
`cobrador_id`	INT UNSIGNED	Sí	→ usuarios.id	Quién recibió el dinero (define la caja y el recaudo)
`monto`	DECIMAL(14, 2)	No		Valor pagado
`metodo`	ENUM	No		efectivo · nequi · daviplata · transferencia · otro · renovacion (no es dinero recibido)
`fecha`	DATETIME	No		Fecha y hora del pago
`nota`	VARCHAR(255)	Sí		Observación del pago
`anulado`	BOOLEAN	No		1 = anulado (no cuenta para saldos ni recaudo)
`motivo_anulacion`	VARCHAR(255)	Sí		Motivo de la anulación
`anulado_por`	INT UNSIGNED	Sí	→ usuarios.id	Usuario que lo anuló
`created_at`	TIMESTAMP	No		Fecha y hora de registro

`gestiones`

Visitas o llamadas sin pago y promesas de pago.

Columna	Tipo	Nulo	Clave / relación	Descripción
`id`	INT UNSIGNED	No	PK · autoincremental	Identificador interno
`credito_id`	INT UNSIGNED	No	→ creditos.id	Crédito gestionado
`usuario_id`	INT UNSIGNED	Sí	→ usuarios.id	Quién hizo la gestión
`tipo`	ENUM	No		visita · llamada · whatsapp
`resultado`	ENUM	No		promesa · no_estaba · sin_dinero · se_niega · direccion_errada · otro
`fecha_promesa`	DATE	Sí		Fecha comprometida (solo promesas)
`monto_promesa`	DECIMAL(14, 2)	Sí		Valor prometido (solo promesas)
`nota`	VARCHAR(255)	Sí		Observación
`created_at`	DATETIME	No		Fecha y hora de registro

`gastos`

Gastos de ruta de los cobradores.

Columna	Tipo	Nulo	Clave / relación	Descripción
`id`	INT UNSIGNED	No	PK · autoincremental	Identificador interno
`cobrador_id`	INT UNSIGNED	No	→ usuarios.id	Cobrador que hizo el gasto
`fecha`	DATE	No		Día del gasto
`concepto`	ENUM	No		transporte · alimentacion · papeleria · recarga · otro
`valor`	DECIMAL(14, 2)	No		Valor del gasto
`nota`	VARCHAR(255)	Sí		Detalle
`anulado`	BOOLEAN	No		1 = anulado (no se descuenta)
`registrado_por`	INT UNSIGNED	Sí	→ usuarios.id	Usuario que lo registró
`created_at`	DATETIME	No		Fecha y hora de registro

`cierres`

Cierres de caja diarios: lo que el cobrador debía entregar y lo que entregó. Uno por cobrador y día.

Columna	Tipo	Nulo	Clave / relación	Descripción
`id`	INT UNSIGNED	No	PK · autoincremental	Identificador interno

Columna	Tipo	Nulo	Clave / relación	Descripción
`cobrador_id`	INT UNSIGNED	No	→ usuarios.id	Cobrador
`fecha`	DATE	No		Día cerrado
`recaudo_efectivo`	DECIMAL(14, 2)	No		Efectivo cobrado ese día (foto al cerrar)
`recaudo_digital`	DECIMAL(14, 2)	No		Cobrado por medios digitales
`gastos`	DECIMAL(14, 2)	No		Gastos del día
`a_entregar`	DECIMAL(14, 2)	No		Efectivo − gastos
`entregado`	DECIMAL(14, 2)	No		Valor recibido (negativo = reembolso al cobrador)
`diferencia`	DECIMAL(14, 2)	No		entregado − a_entregar (negativo = faltante)
`observaciones`	VARCHAR(255)	Sí		Nota del cierre
`recibido_por`	INT UNSIGNED	Sí	→ usuarios.id	Administrador que recibió
`created_at`	DATETIME	No		Fecha y hora de registro

`configuracion`

Parámetros editables (datos de la empresa y valores por defecto de los créditos).

Columna	Tipo	Nulo	Clave / relación	Descripción
`clave`	VARCHAR(60)	No	PK	empresa_nombre, empresa_nit, empresa_telefono, empresa_direccion, recibo_mensaje, interes_defecto, cuotas_defecto, frecuencia_defecto
`valor`	TEXT	Sí		Valor del parámetro

`auditoria`

Bitácora de seguridad y operaciones.

Columna	Tipo	Nulo	Clave / relación	Descripción
`id`	INT UNSIGNED	No	PK · autoincremental	Identificador interno
`usuario_id`	INT UNSIGNED	Sí	→ usuarios.id	Usuario que realizó la acción
`accion`	VARCHAR(40)	No		Código del evento (login_ok, pago_anulado, caja_cerrada…)
`detalle`	VARCHAR(500)	Sí		Descripción del evento
`entidad`	VARCHAR(30)	Sí		Tipo de registro afectado
`entidad_id`	INT UNSIGNED	Sí		Id del registro afectado
`ip`	VARCHAR(45)	Sí		Dirección IP de origen
`user_agent`	VARCHAR(255)	Sí		Navegador de origen

Columna	Tipo	Nulo	Clave / relación	Descripción
`created_at`	DATETIME	No		Fecha y hora de registro

`intentos_login`

Intentos de ingreso para el bloqueo por fuerza bruta (se depuran a los 30 días).

Columna	Tipo	Nulo	Clave / relación	Descripción
`id`	INT UNSIGNED	No	PK · autoincremental	Identificador interno
`usuario`	VARCHAR(120)	No		Usuario o correo digitado
`ip`	VARCHAR(45)	No		IP de origen
`exito`	BOOLEAN	No		1 ingreso correcto · 0 fallido
`created_at`	DATETIME	No		Fecha y hora de registro

`operaciones_offline`

Identificador único de cada cobro o gestión hecho desde la ruta, para no registrarlo dos veces si llega repetido.

Columna	Tipo	Nulo	Clave / relación	Descripción
`uuid`	CHAR(36)	No	PK	Identificador generado en el celular
`usuario_id`	INT UNSIGNED	Sí	→ usuarios.id	Usuario que la registró
`tipo`	ENUM	No		pago · gestion
`entidad_id`	INT UNSIGNED	Sí		Pago o gestión creada
`creado_en`	DATETIME	Sí		Hora del celular al registrarla
`sin_conexion`	TINYINT(1)	No		1 = se guardó sin internet y llegó al sincronizar
`created_at`	DATETIME	No	Índice (usuario_id, created_at)	Hora en que llegó al servidor